

LABORATORY OBSERVATION/RECORD

Course:	Full Stack Web Development
Course Code:	23CM4121
Observation Task No.:	2
Student Name:	P. Dharani
Roll No.:	A24126552261
Date:	28/7/26

1. TITLE

Working with CSS3 and HTML5 Canvas Graphics

2. AIM

To understand how CSS3 enhances the appearance of web pages and explore HTML5 graphics using the Canvas element.

3. OBJECTIVE

The objectives of this observation task are:

- To study page visual presentation enhancement using CSS3 styling structures.
- To apply border styles, rounded corners (border-radius), and drop shadows (box-shadow).
- To implement Flexbox container models for layout positioning and grid wraps.
- To utilize the HTML5 <canvas> tag to render inline 2D raster vector drawings.
- To write scripting logic executing paths, shape fills, outlines, and text strokes on canvas.

4. THEORY

CSS3 (Cascading Style Sheets, Level 3) extends the visual presentation layer of HTML pages, offering rules to align, format, and animate elements. Flexbox organizes card blocks in container alignments, while drop shadows and border corners add layers of depth. HTML5 introduces graphics natively via the <canvas> element, which exposes a 2D rendering context. JavaScript scripts target this canvas API to programmatically draw rectangles, lines, arcs, and text on a grid coordinate layout.

Theory of CSS3 Layouts & HTML5 Canvas:

- CSS3 Selectors & Transitions: Box elements are styled using background-color, border-radius, and box-shadow. Interactive hover transitions are applied using the transition shorthand.
- CSS3 Flexbox: Enforces standard container alignments (display: flex) dynamically adjusting spacings (gap) and wrapping grid elements.
- HTML5 Canvas Element: A raster graphics container. The context object (canvas.getContext('2d')) exposes drawing commands.
- 2D Context Drawing Operations: Rectangles are drawn using fillRect() and strokeRect(). Paths are initiated with beginPath(), mapped with moveTo() and lineTo(), and drawn with stroke(). Arcs are drawn via arc().

5. ALGORITHM / PROCEDURE

- 1 Establish project directories: Observation/ObTask2/html and Docs.
- 2 Create canvas.html to implement graphic scripts.
- 3 Embed a <canvas> tag with custom dimensions and borders.
- 4 Retrieve the 2D canvas context and execute fillRect, strokeRect, and line path strokes.
- 5 Create partB.html to implement CSS3 enhancements.
- 6 Construct a layout containing headers, nav sections, and a flex container hosting three card blocks.
- 7 Apply CSS rules: flex displays, card dimensions, margins, rounded corners, box shadows, and hover transitions.
- 8 Open files in browser, verifying CSS hover properties, Flexbox wrap alignments, and Canvas graphics rendering.

6. PROGRAM

Files: canvas.html and partB.html (Complete)

canvas.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Styled WebPage</title>
</head>
<body>
<center>
<canvas
  height="480px"
  width="960px"
  id="artBoard"
  style="border: 1px solid black; border-radius: 6px;">
</canvas>
</center>
</body>
<script>
  const artBoard = document.getElementById("artBoard");
  const pen = artBoard.getContext("2d");

  // Filled rectangle
  pen.fillStyle = "teal";
  pen.fillRect(30, 30, 160, 180);

  // Outlined rectangle
  pen.strokeStyle = "orange";
  pen.lineWidth = 4;
  pen.strokeRect(220, 30, 160, 90);

  // Straight line path
  pen.beginPath();
  pen.moveTo(60, 280);
  pen.lineTo(220, 280);
  pen.strokeStyle = "purple";
  pen.lineWidth = 4;
  pen.stroke();

  // Filled circular arc
  pen.beginPath();
  pen.arc(330, 190, 45, 0, Math.PI * 2);
  pen.fillStyle = "black";
  pen.fill();

  // Text rendering
  pen.beginPath();
  pen.font = "14px Arial";
  pen.fillStyle = "black";
  pen.fillText("KING CYCLONE BY SRIKAR", 480, 230);
</script>
</html>
```

partB.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Styled</title>
<style>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: Arial, sans-serif;
}

#navbar {
  background-color: #1e3a8a;
  height: 45px;
  color: white;
}

ul {
  list-style: none;
  display: flex;
  height: 100%;
  gap: 18px;
  align-items: center;
  justify-content: flex-end;
  padding-right: 20px;
}

.banner {
  width: 100%;
  height: 100px;
  background-color: #111827;
  color: white;
  text-align: center;
  padding-top: 30px;
}

.cardGrid {
  display: flex;
  gap: 25px;
  padding: 20px;
  justify-content: center;
}

.infoCard {
  width: 480px;
  height: 290px;
  border: 2px solid #111827;
  border-radius: 12px;
  padding: 28px;
  text-align: center;
  justify-content: center;
  box-shadow: 2px 2px 10px rgba(0, 0, 0, 0.15);
  background-color: #5cb85c;
  font-size: 20px;
  display: flex;
  flex-direction: column;
  gap: 14px;
}

.actionBtn {
  width: 110px;
  height: 38px;
  border: none;
  border-radius: 5px;
  background-color: #1e3a8a;
  color: white;
  cursor: pointer;
  transition: all 0.4s ease;
}

.actionBtn:hover {
  background-color: rgb(0, 0, 0);
  box-shadow: 0px 0px 18px 8px rgb(0, 229, 255);
  transform: translateY(-3px);
}
```

```

    li {
      cursor: pointer;
      transition: all 0.4s ease;
    }

    li:hover {
      color: rgb(0, 229, 255);
      transform: translateY(-3px);
    }
  }
</style>
</head>
<body>
<header>
  <div class="banner">
    <h1>WELCOME TO PAGE</h1>
  </div>
  <nav id="navbar">
    <ul>
      <li>HOME</li>
      <li>EXPLORE</li>
      <li>SERVICES</li>
      <li>LOCATIONS</li>
      <li>CONTACT</li>
    </ul>
  </nav>
</header>

<main class="cardGrid">
  <article class="infoCard">
    <h2>Feature One</h2>
    <p>
      This is the first content card. It highlights a bold
      heading style to stand out from the surrounding body text.
    </p>
    <center><button class="actionBtn">Read More</button></center>
  </article>

  <!-- Card 2 -->
  <article class="infoCard">
    <h2>Feature Two</h2>
    <p>
      This is the second content card. Notice the soft drop
      shadow and rounded corners applied to the card frame.
    </p>
    <center><button class="actionBtn">Read More</button></center>
  </article>

  <!-- Card 3 -->
  <article class="infoCard">
    <h2>Feature Three</h2>
    <p>
      This is the third content card. Hover the button below or
      the top navigation links to see the color change instantly.
    </p>
    <center><button class="actionBtn">Read More</button></center>
  </article>
</main>
</body>
</html>

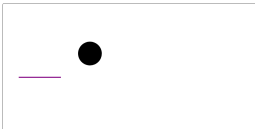
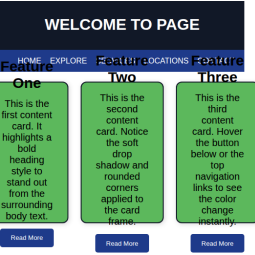
```

7. CODE EXPLANATION

Code Selector / Statement	Functional Description
<code>artBoard.getContext("2d")</code>	Retrieves the 2D rendering graphics drawing context bound to the canvas element.
<code>pen.fillRect(30,30,160,180)</code>	Draws a solid filled rectangle at x=30, y=30 with width=160 and height=180.
<code>pen.strokeRect(220,30,160,90)</code>	Draws a rectangle outline using the active stroke color and line width.
<code>display: flex; gap: 25px;</code>	Enables Flexbox layout, aligning cards side-by-side with 25px spacing.
<code>box-shadow: 2px 2px 10px rgba(...)</code>	Applies a soft transparent drop shadow effect underneath each feature card.

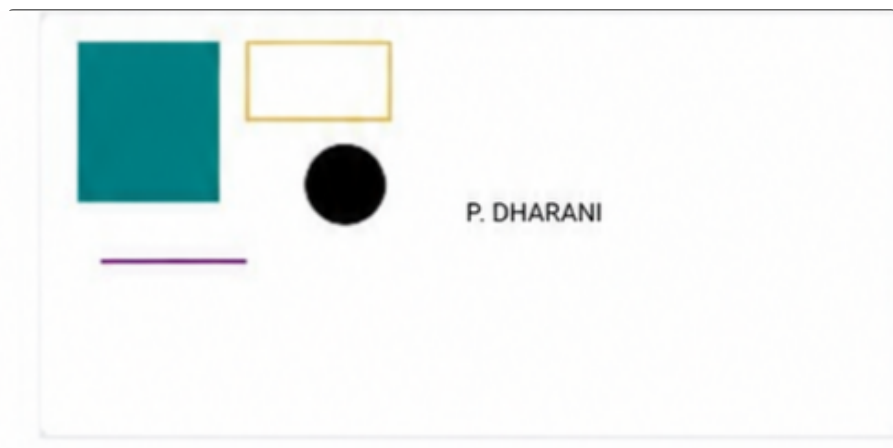
8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test executions below evaluate CSS3 layouts and Canvas methods in Chrome, including 1 negative test case demonstrating a responsiveness defect:

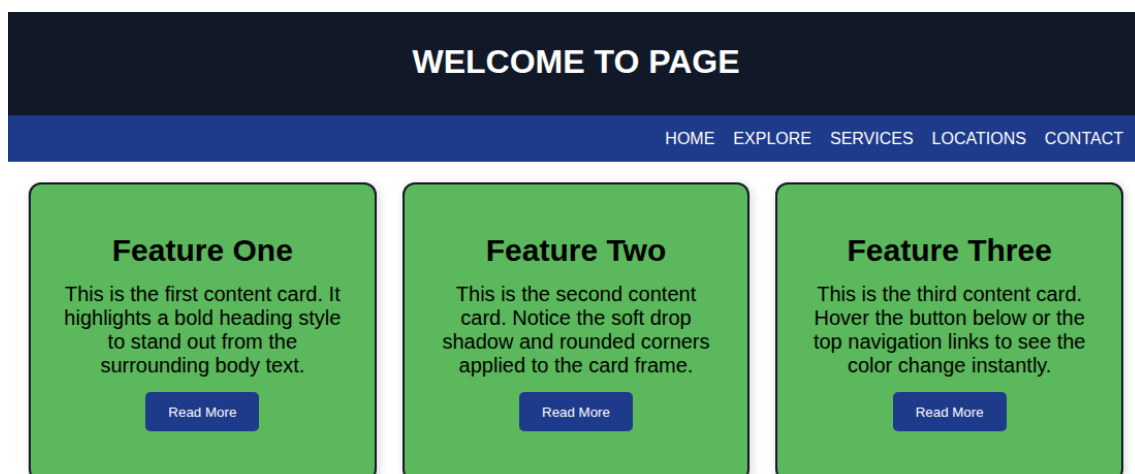
TC ID	Test Description	Input / Action	Expected Result	Screenshot	Status
TC-01	Canvas Shapes Fills	Execute fillRect & strokeRect	Teal filled rectangle and orange rectangle outline render at offsets		Pass
TC-02	Canvas Arc & Paths	Execute lineTo & arc	Purple line and solid black circle render on canvas coordinates		Pass
TC-03	Canvas Text Typography	Execute fillText()	String 'P. Dharani' displays at x=480,y=230		Pass
TC-04	Header & Nav Alignment	Inspect partB.html navigation	Dark title banner with right-aligned flex navbar displays cleanly		Pass
TC-05	Mobile Flexbox Wrapping	Resize window to 500px viewport	Cards should wrap vertically; instead they stay in a single row and get compressed into narrow, unreadable columns because flex-wrap: wrap is missing		Fail

9. OUTPUT

Output 1: HTML5 Canvas Elements Rendering (canvas.html)



Output 2: Enhanced CSS3 Card Grid Layout (partB.html)



10. RESULT

Thus, page visual presentation styles using CSS3 (selectors, flex models, borders, box-shadows, animations) and HTML5 inline vector canvas graphics drawings were successfully designed, implemented, and verified using the test cases above, accompanied by verified execution screenshots and responsiveness defect analysis.

Submission Package Structure:

```
Observation/ObTask2/
|-- Docs/
|   |-- ObsWeek2.pdf
|-- Screenshots/
|   |-- canvas_output.png
|   |-- cardgrid_output.png
|   |-- tc01_canvas_rects.png
|   |-- tc02_canvas_paths.png
|   |-- tc03_canvas_text.png
|   |-- tc04_header_nav.png
|   |-- tc05_flex_overflow_fail.png
-- html/
   |-- canvas.html
```

-- partB.html